

Xor filter for Redis

Aleksandr Goncharenko
Ca'Foscari

913215@stud.unive.it

Abstract

Redis is a high-performance in-memory key-value storage used for applications in order to ensure fast data access and low latency. Redis contains different probabilistic data structures, among which there are two filters: Bloom filter and Cuckoo filter, which are dynamic, that allows to build them on fly. Meanwhile, the static filters, which require access to the whole data during the build stage, are generally faster and consume less memory, but are still not implemented for Redis. In this paper, we implement and analyze the XOR filter for Redis.

1. Introduction

Redis[1] is a widely used in-memory database whose main goal is providing fast data access and low-latency operations. Besides that, it also provides probabilistic data structures, including two filters(Bloom and Cuckoo), which allow executing approximate membership queries with zero false negative rate and a tunable false positive rate.

Bloom[2] and Cuckoo[3] filters are dynamic structures, which could be built on the fly. However, that flexibility comes with additional memory cost. The latest theoretical lower bound for memory of dynamic filters[4] is $m \log_2(\delta^{-1}) + m \log_2(e) - o(m)$ while static filters allow us to use lower memory - $m \log_2 \delta^{-1} - O(1)$. None of the static filters is presented in Redis, while for lots of use cases it can be more efficient than dynamic filters. In this paper, we describe the implementation and results of a comparison of our XOR implementation and Redis' Bloom filter and Cuckoo implementations. The main contributions of this work are:

- The implementation of an XOR filter for Redis.
- An experimental comparison of XOR, Bloom, and Cuckoo filters.
- An analysis of the observed results.
- Optimization of queue size for the XOR-like filters.
- An implementation of the XOR4 variation, which is less commonly supported by existing XOR filter libraries.

2. Approximate Membership Filters Evolution

2.1. Bloom filter

The evolution of approximate membership algorithms starts with the Bloom filter[2]. That filter provides a probabilistic data structure which represent a set with a flexible false-positive probability. It uses multiple hash functions and a bit array. A key property is the absence of false negative results, while false positive results could happen. It uses $1.44m \log(\delta^{-1})$ bits, where δ is a FPR.

2.2. XOR Filter

The first significant step towards modern static filters was made by Dietzfelbinger and Pagh[5]. They were studying compact representation of static sets based on hashing and the XOR operation. The key idea of their work was to associate each key with several positions in an array and assign values of those positions such XOR of those cells would be equal to a fingerprint of the key. In the paper Xor Filters: Faster and Smaller Than Bloom and Cuckoo Filters[6], the implementation and benchmarks of that algorithm were evaluated for the first time. It uses $1.23m \log(\delta^{-1})$ bits, where δ is a FPR.

2.3. Binary Fuse Filter

The next development was the Binary Fuse filter[7]. That filter is an evolution of the XOR filter, which decrease memory usage by segmenting the array into overlapping regions and restricting the key's hash location to a specific region. That creates a "fused" hypergraph structure where the peeling phase is more predictable and allows decreasing array size without increasing the building phase failure rate. Moreover, it increases cache locality, making lookup faster. It uses $1.125m \log(\delta^{-1})$ bits, where δ is a FPR.

2.4. ZOR Filter

The most recent development is the ZOR filter[8]. It replaces restart-on-failure with deterministic peeling, which discards a small number of keys and stores them in an additional compact structure. They show that the fraction of abandoned keys is less than 1% for a number of hash func-

tions greater than 4. It uses $1.01m\log(\delta^{-1})$ bits, where δ is a FPR.

3. Theoretical Analysis

3.1. Membership test

The membership test function calculates 3 hash functions h_1, h_2, h_3 and reconstruct expected fingerprint from entries of an array:

$$fp_{expected} = A[h_1(x)] \text{ XOR } A[h_2(x)] \text{ XOR } A[h_3(x)]$$

Then this value is compared with an actual fingerprint, and if they are equal, then it returns true; otherwise, it returns false. The time complexity is constant, since it is necessary to compute the fingerprint and 3 hashes, make 3 memory accesses, and compute XOR twice.

3.2. Building phase

The build phase consists of two phases: Mapping(peeling) and Assigning, and follows the algorithm from [9].

3.2.1. Mapping

We initialize an array H of sets of keys with size $c|S|$, where $|S|$ is the set of objects, and c is a coefficient. Then each key x is hashed three times to $h_1(x), h_2(x), h_3(x)$. Then, repeatedly, we pick a set with only one key, append the key to stack σ , and remove the key from its three locations. That process is repeated until we get into a situation where we do not have any sets with only one key. It can happen if we proceed through all sets or if the algorithm gets stuck. In the second case, we rerun the algorithm with the other seed. Larger values of the parameter c decrease the probability of a rerun. The time complexity of that step is $O(n)$.

3.2.2. Assigning

During the assigning step, we pick elements x from the σ and iteratively fill the result array B with a size $c|S|$:

$$B[i] \leftarrow fp(x) \text{ XOR } B[h_1(x)] \text{ XOR } B[h_2(x)] \text{ XOR } B[h_3(x)]$$

The time complexity of that step is $O(n)$.

3.3. Critical point

Xor filter with 3 hash functions can be modeled as a random 3-hypergraph, where cells are vertices and keys are hyperedges, which connect 3 vertices. During the peeling phase, we remove keys that fall into only one cell. However, it can get stuck if during the peeling stage cells stop being removed - in other words, when a new iteration coincides with an old one. It happens if we get a dense 2-core sub-hypergraph where each vertex has a degree larger than one. In order to maintain that, we allocate more cells than the initial number of data samples. In that section, we provide simplified proof [10] that it is enough to allocate $c = 1.23N$ cells.

Lets say x - ratio of "alive" cells, N is a number of keys and m is a number of cells, therefore $c = \frac{m}{N}$ and $\alpha = c^{-1}$. Since each key connects 3 vertices, the average number of hyperedges incident to a vertex is $\lambda = \frac{3N}{m} = 3\alpha$, since there are three hash functions. A hyperedge remains active if and only if its corresponding vertices are active, which occurs with probability p^2 , where p is the probability that a vertex is active. As a result, the average degree of active hyperedges is $\mu = 3\alpha x^2$. Under the Poisson assumption, the probability that a vertex is peeled is $e^{-3\alpha x^2}$.

The peeling stuck if $x_{i+1} = x_i$, and then $x = 1 - e^{-3\alpha x^2}$. In order to find α , we can solve this equation for c as a function of x :

$$c(x) = \frac{3x^2}{-\ln(1-x)}$$

. In order to ensure that peeling can proceed for any values of x from 0 to 1, we have to pick c such that it is greater than the maximal value of $c(x)$. After computing the derivative, making it equal to zero, and solving, we obtain $x^* \approx 0.71533$ - the point where $c(x)$ is maximal and $c(x^*) \approx 1.22$. That value is usually rounded to 1.23 to provide margin for finite hypergraphs.

3.4. Error rate

Let's say we have a fingerprint with f bits. During the membership test, we calculate fingerprint fp . Under the assumption that we have three ideal random hash functions, every one of 2^f possible fingerprints is equiprobable. For the false positive result, there should be a random coincidence of fingerprints. The probability of that is $P(FP) = \frac{1}{2^f}$. The false negative rate is zero, since during the building phase we write the fingerprints to the cells such that the XOR of the three cells reconstructs the original fingerprint.

4. Implementation Details

4.1. Xor4

Since there is no 4-bit datatype in C-language, it is not possible to directly implement the XOR4 filter. In order to maintain that fingerprint size, we pack two 4-bit fingerprints in one uint8 variable. In those settings, peak usage of memory during the build phase would be the same as for XOR8; however, the final data structure size would be as if we had uint4.

4.2. Queue size analysis

During the XOR building phase we use a queue in which we store 1-degree buckets. There are two straightforward ways to set up queue capacity. The first is to set up a maximum size of the queue large enough to maintain all M buckets and never resize it. Although this approach allows us to avoid costly reallocation, it consumes a lot of memory. The

second approach is to allocate a small initial queue and re-size during overflows. However, that approach leads to additional overhead.

In our implementation, we use a dynamically growing queue with an initial capacity of $0.4M$, which almost never would be resized. We choose that value based on the expected number of 1-degree buckets in an XOR-hypergraph. That constant can be used for other XOR-like algorithms, such as binary fuse or ZOR.

For a random XOR-hypergraph, the degree of any bucket is a random variable with a Poisson distribution. $D \sim \text{Poisson}(\lambda)$. Then, $P(D = 1) = \lambda e^{-\lambda}$ and expected size of the queue is $E[Q(\lambda)] = M\lambda e^{-\lambda}$ with a maximum when $\lambda = 1$. Then number of degree-1 buckets is a binomial $Y \sim \text{Binomial}(M, e^{-1})$ with $\mu = \frac{M}{e}$ and $\sigma = \sqrt{\frac{M}{e}(1 - e^{-1})}$. For a large M , we can approximate this with a normal distribution. Then we can invent a function $Q_{cap}(M, r)$ - size of queue for a hypergraph with M buckets which will not overflow with a probability p :

$$Q(M, r) = \frac{M}{e} + \Phi^{-1}(1 - p) \sqrt{\frac{M}{e}(1 - e^{-1})}$$

For a probability 10^{-12} and large hypergraph ($> 10^4$ original elements) $Q_{cap} < 0.4M$. As a result, during the building phase, it saves $0.6 * 1.23 * 32 = 23.6$ bits per original element.

During our benchmarks, the queue was never reallocated. However, the heuristic should be further validated across a larger range of configurations.

4.3. Pair packing

In our implementation, we use bit fields for the pair structure. The structure stores the element index in 30 bits and a slot number in two bits. The slot identifies which of the hash positions corresponds to the bucket. Since we have only 3 hash functions, 2 bits is enough. As a result, instead of $8+32 = 40$ bits (if we use structure packing), we use only 32 bits, which saves us 8 bits per original element; however, it limits the maximum number of elements to 2^{30}

4.4. Bucket

Instead of storing sets of key indices, we use a bucket data structure. It has an XOR sum and a counter for each bucket. The XOR sum allows us to recover the index of the key if the bucket contains a single element and remove elements from the bucket, while the counter is used to calculate the degree of the bucket.

4.5. Redis Module API

The XOR filter implementation has the following API

- `XOR.BUILD <dest_key> <source_set> <fp_bits>`: Constructs a XOR filter in `dest_key`

by scanning elements from an existing Redis SET (`source_set`) using batch SSCAN iterations. The `fp_bits` parameter sets the fingerprint precision (4, 8, 16, or 32 bits).

- `XOR.EXISTS <filter_key> <item>`: Check the membership for a given `item` in $O(1)$ constant time. Returns 1 if present (with probable false positives) or 0 if definitely not present.
- `XOR.INFO <filter_key>`: Returns metadata dictionary for the filter.

5. Experimental Results

5.1. Dataset

To evaluate our implementation, we use the Weakpass 4[11] wordlist. It is one of the largest password wordlists, which aggregates data for password-cracking and security-testing research. The dataset contains 10 billion commonly used and previously leaked passwords.

5.2. Experimental settings

In our experiments, we compare 3 families of filters: XOR, Bloom, and Cuckoo. For Bloom and Cuckoo, we use a Redis implementation, while the XOR filter is implemented by us.

All experiments were conducted on a Windows laptop with i7-12700H processor in Docker containers with limits of 24 GB of memory.

We evaluate the XOR filter with fingerprint sizes 4, 8, 16, 32 bits, the Bloom filter with numbers of hash functions 4, 8, 16, 32, and the Cuckoo filter with bucket sizes 1, 2, 4, 8. We include the Cuckoo filter for completeness; however, its results should be interpreted with caution, since its parameters are managed by the Redis implementation: the number of filters and buckets cannot be fixed with the provided API, making a comparison of algorithms less straightforward. Therefore, our analysis focuses only on the XOR and Bloom filters, while the Cuckoo filter results are presented as a reference. We compare algorithms on subsets of the Weakpass dataset with sizes 10 and 100 million. Since Redis has a significant network overhead per request, we put all data in Redis in one request instead of adding elements one by one.

For the building phase, we evaluate the final filter size, peak memory consumption (from `memory.peak` in Docker), and the build time.

For evaluating latency metrics, we use 10^6 items that were fitted into the filter and 10^6 items which was not. The data is divided into batches with size 1000. Inside each batch, objects are queried sequentially (one at a time). The total time required to process one batch is measured and divided by the number of objects inside the batch. The first 10% of batches are used as a warmup. The resulting

per-query latency measurements are then used to calculate the mean, standard deviation, minimum, maximum, and the 50th, 90th, 95th, and 99th percentiles.

In order to evaluate the false positive rate, we use 10^7 original objects.

5.3. Building cost analysis

During the analysis, we found some interesting facts (Fig. 1):

- For each dataset size, the peak memory consumption is almost the same across configurations within the XOR filter and, separately, within the Bloom filter. The main reason is that the memory required to load and process the dataset dominate over filter sizes themselves are relatively small compared to this overhead.
- The throughput of the XOR filter is the smallest and decreases from 0.5M elements/s for the 10M subset to 0.3M elements/s for the 100M subset. It is caused by the increasing overhead of the XOR filter: larger datasets process a larger hypergraph and associated structures.

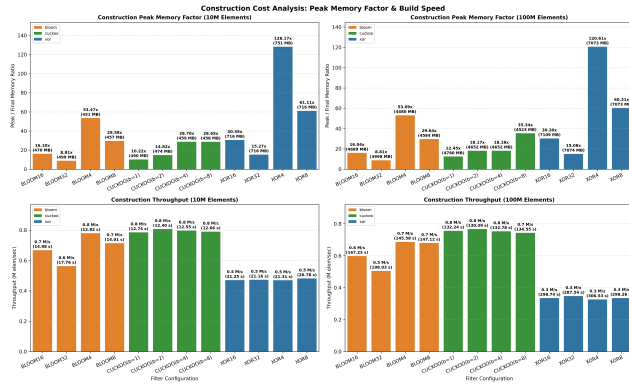


Figure 1. Building cost analysis

5.4. FPR

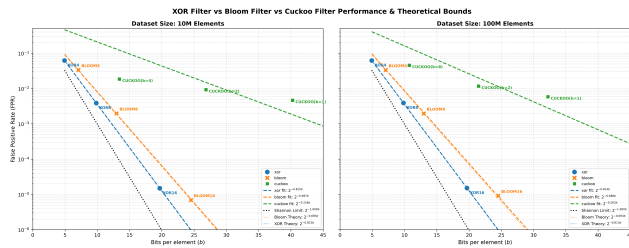


Figure 2. FPR Performance and theoretical bounds

The analysis of the filter quality revealed several additional observations (Fig. 2):

- The XOR filter with 32 bits and the Bloom filter with 32 hashes has a zero false positive empirical results, since

FPRs for them are very small and require subset larger than 10M.

- The XOR filter provides better results than the Bloom filter and Cuckoo filter in terms of FPR/Bits per element.
- The theoretical and empirical FPR and Bits per element coincide.

5.5. Latency

The analysis of latency (Fig. 3) shows that latency for all filters are almost the same and dominated by system overhead. This is also confirmed by the ping latency, which has almost same latency as filter's membership test. However there are some cases where latency is abnormal large, which, possibly, was caused by system factors such as CPU scheduling (or thermal throttling, since thermal paste on the laptop used for tests is old), or other background processes rather than by the filter itself.

The main insight is that it is not necessary to optimize speed of lookup request, since it can not significantly increase speed, because of Redis overhead.

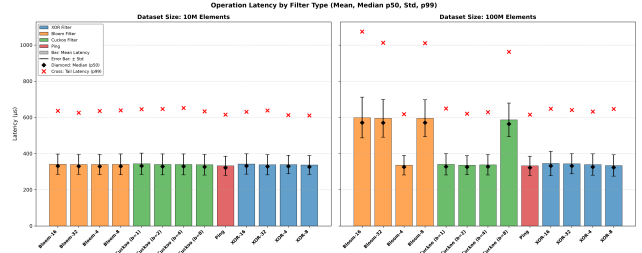


Figure 3. Lookup latency. The Ping is a latency of the ping Redis command command.

6. Conclusion

In this paper, we introduced an implementation of XOR filters, including a custom 4-bit packed variant (XOR4), for Redis. Existing filters in Redis offer dynamic updates; however, they have significant memory overheads. Our goal was to provide a more space-efficient alternative for approximate membership queries. We introduced several memory optimizations for Redis: more efficient queue initial capacity bounded at 0.4M (it saves 23.6 bits per element during the build phase), bit-field pair representations, and a 4-bit version of XOR filter. Our experiments use up to 100 million items from the Weakpass 4 dataset confirmed that XOR filters achieve a better False Positive Rate relative to bit consumption compared to Bloom and Cuckoo implementations.

The main trade-off is construction cost: the building phase requires hypergraph processing, which causes lower build throughput (0.3–0.5M elements/s) and memory allocation overhead. However, once constructed, the $O(1)$

membership query, requiring only three hash computations and bitwise operations, ensures fast lookup.

In future work, we are going to implement more compact filters, such as Binary Fuse and ZOR filters, making memory consumption closer to the theoretical lower bound.

References

- [1] Redis Ltd., “Redis.” [Online]. Available: <https://github.com/redis/redis> 1
- [2] B. H. Bloom, “Space/time trade-offs in hash coding with allowable errors,” *Communications of the ACM*, vol. 13, no. 7, 1970. 1
- [3] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, “Cuckoo filter: Practically better than bloom,” in *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies*. ACM, 2014. 1
- [4] W. Kuzmaul, J. Liang, and R. Zhou, “Fingerprint filters are optimal,” *arXiv preprint arXiv:2510.18129*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.18129> 1
- [5] M. Dietzfelbinger and R. Pagh, “Succinct data structures for retrieval and approximate membership,” in *Automata, Languages and Programming*. Springer, 2008. 1
- [6] T. M. Graf and D. Lemire, “Xor filters: Faster and smaller than bloom and cuckoo filters,” *arXiv preprint arXiv:1912.08258*, 2020. [Online]. Available: <https://arxiv.org/abs/1912.08258> 1
- [7] —, “Binary fuse filters: Fast and smaller than xor filters,” *arXiv preprint arXiv:2201.01174*, 2022. [Online]. Available: <https://arxiv.org/abs/2201.01174> 1
- [8] A. Limasset, “Zor filters: fast and smaller than fuse filters,” *arXiv preprint arXiv:2602.03525*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.03525> 1
- [9] F. C. Botelho, R. Pagh, and N. Ziviani, “Simple and space-efficient minimal perfect hash functions,” in *Proceedings of the 10th International Conference on Algorithms and Data Structures (WADS'07)*. Berlin, Heidelberg: Springer-Verlag, 2007. 2
- [10] Z. J. Czech, G. Havas, and B. S. Majewski, “Fundamental study of perfect hashing,” *Theoretical Computer Science*, vol. 182, 1997. 2
- [11] Weakpass, “Weakpass 4 password wordlist.” [Online]. Available: <https://weakpass.com/> 3